

รายงานความคืบหน้า Raycaster

โปรเจกต์นี้วาดภาพมุมมองบุคคลที่หนึ่งจากแผนที่ตาราง 2 มิติ (สไตล์ Wolfenstein 3D) โดยใช้ Linear Algebra เป็นแกนหลัก ทิศทางของผู้เล่นคือ vector กล้องคือ matrix 2x2 การหมุนคือการคูณ rotation matrix และ perpendicular distance ได้จาก dot product

รายงานนี้เล่าความคืบหน้าที่ได้ศึกษาและพัฒนา

ปัญหา: คำนวณทุก pixel ซ้ำเกินไป

วิธีที่ตรงที่สุดในการวาดห้อง 3 มิติคือให้ทุกพิกเซลของจอมีรังสีของตัวเอง แล้วเดินตามรังสีไปที่ละก้าวเล็กๆ จนชนกำแพง

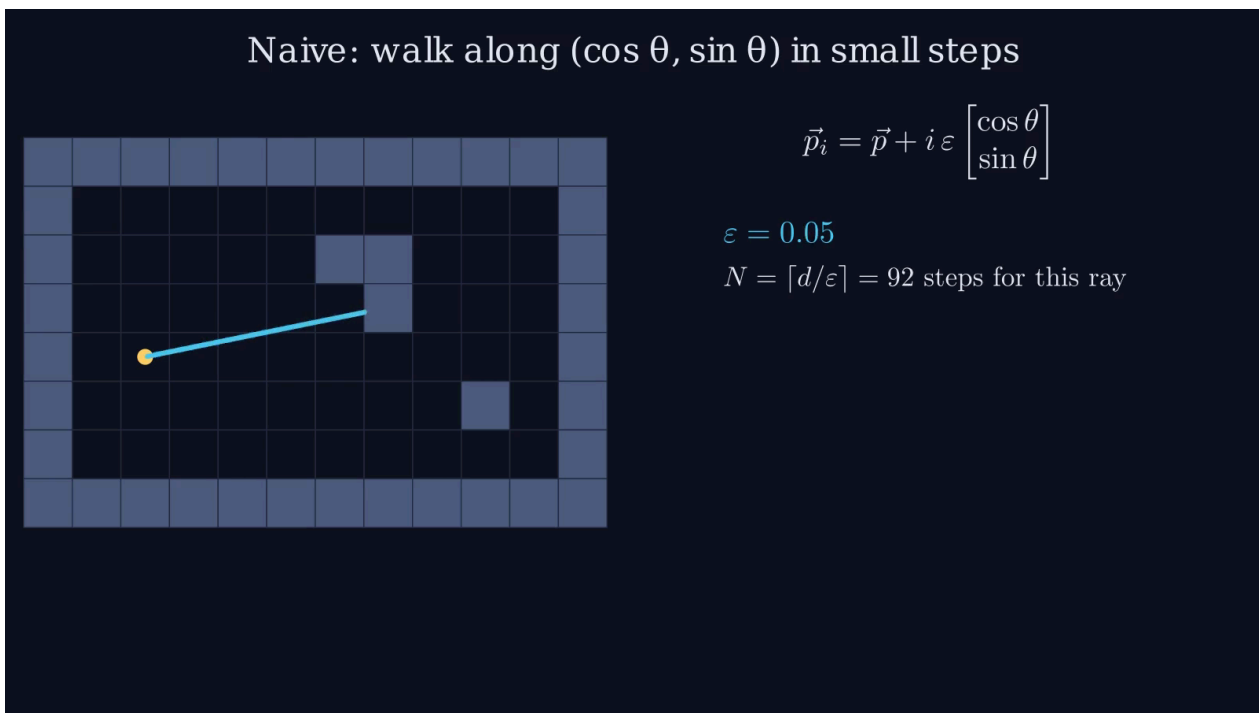
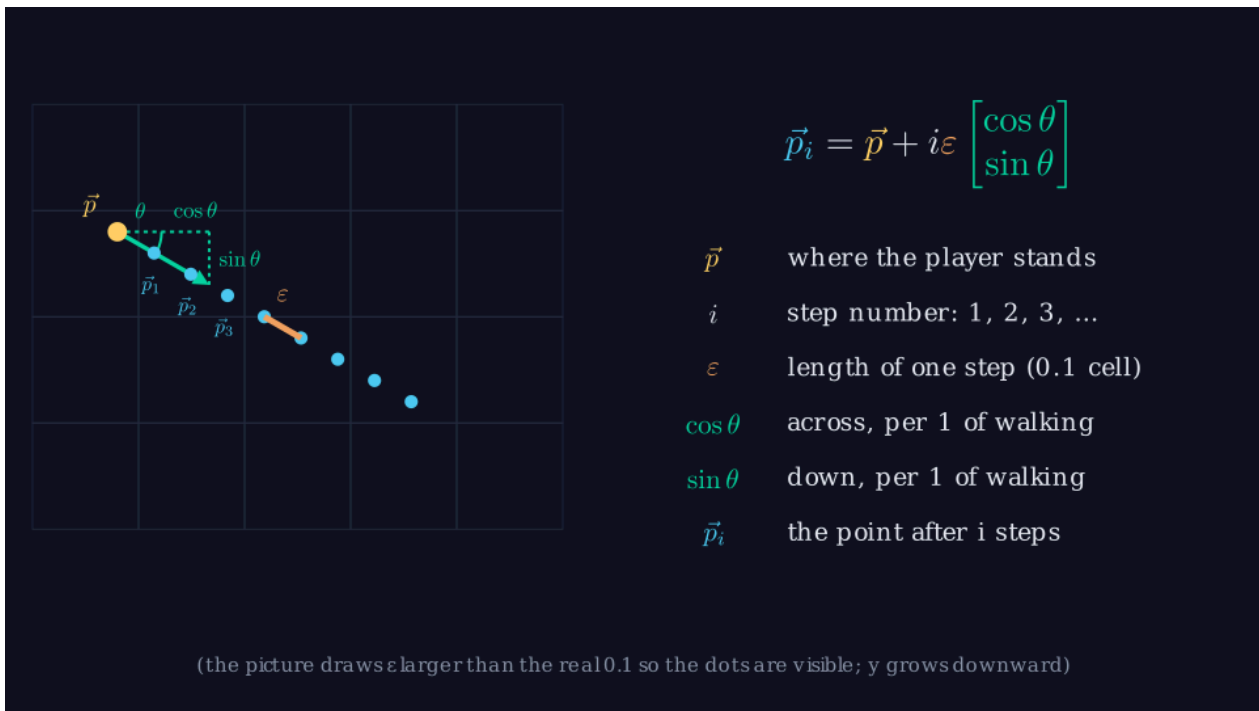
ภาพหนึ่งภาพมี $320 \times 200 = 64,000$ พิกเซล รังสีหนึ่งเส้นเริ่มจากตำแหน่งผู้เล่น p ไปที่มุม θ และเดินทีละก้าวยาว ϵ (เช่น 0.1 ช่อง) หลังเดิน i ก้าวจะอยู่ที่

$$\vec{p}_i = \vec{p} + i \epsilon \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}$$

ความหมายของแต่ละตัวแปร:

- p_i : ตำแหน่งของรังสีหลังเดิน i ก้าว (vector)
- p : ตำแหน่งผู้เล่น คือจุดเริ่มต้นของรังสี (vector)
- i : จำนวนก้าวที่เดินไปแล้ว (0, 1, 2, ...)
- ϵ : ความยาวต่อก้าว เช่น 0.1 ช่อง
- θ : มุมของรังสี วัดจากแกน x
- $\langle \cos \theta, \sin \theta \rangle$: vector หนึ่งหน่วยที่ชี้ไปทางมุม θ

ทุกก้าวถามว่าจุดนี้อยู่ในกำแพงหรือยัง ก้าวแรกที่ใช้คือกำแพงที่พิกเซลนั้นเห็น แต่ละก้าวคือการบวก vector เล็กๆ เดิมซ้ำ (บวกสองครั้ง: แกน x และแกน y)



ในห้องตัวอย่าง กำแพงอยู่ห่างเฉลี่ย 5.81 ช่อง ซึ่งคือ 59 ก้าวต่อเส้นที่ $\varepsilon = 0.1$ จำนวนงานต่อหนึ่งภาพจึงเป็น

64,000 lines $\times$ 59 steps $\times$ 2 additions $\approx$ 7.6 million additions

บวก sin และ cos อีกหนึ่งคู่ต่อเส้น ตัวเลขเหล่านี้เป็นทศนิยม ซึ่งคอมพิวเตอร์ปี 1992 คำนวณได้ช้ามาก เครื่อง 386 ความเร็ว 33 MHz จะใช้ประมาณ 6 วินาทีต่อหนึ่งภาพ แต่เกม (Wolfenstein 3D) ต้องการ 70 ภาพต่อวินาที คือภาพละ 14 มิลลิวินาที ช้า

กว่าที่ต้องการราว 418 เท่า (ตัวเลขเป็นการประมาณจากจำนวนคำสั่งทศนิยม ไม่ใช่การวัดจริง)

ทางแก้: ทำอะไรได้บ้าง และ Linear Algebra ช่วยตรงไหน

เราแก้ได้ด้วยการจำกัดโลกให้ง่ายลงและเปลี่ยนจากมุมเป็น vector

ข้อจำกัดมี 3 ข้อ:

- แผนที่เป็นตาราง 2 มิติ ช่องละ 1×1 และแต่ละช่องเป็น 0 (พื้นว่าง) หรือ 1 (กำแพง)
- กำแพงตั้งตรงและสูงเท่ากันทุกแห่ง พื้นและเพดานเป็นสี่เหลี่ยม
- ผู้เล่นหันซ้ายขวาเท่านั้น ไม่เงยหรือก้ม

ภายใต้ข้อจำกัดนี้ ทั้งคอลัมน์ของจอเห็นกำแพงแผ่นเดียวที่ระยะเดียว จึงยิงรังสีเพียง 1 เส้นต่อคอลัมน์ คือ 320 เส้นแทน 64,000 เส้น (น้อยลง 200 เท่า)

ปัญหา	ทำอย่างไร	Linear Application	หัวข้อ
1 pixel 1 เส้นรังสี มากเกินไป (64,000 เส้น)	1 เส้นต่อคอลัมน์ของจอ (320 เส้น)	vector 2 มิติ: รังสี = $d + c \cdot P$ (linear combination ของ vector สองตัว)	1
มีการคำนวณ sin และ cos ทุกเส้น (floating point)	บอกทิศของรังสีด้วย vector แทนมุม	matrix $M = [d \ P]$, $r = M \cdot \langle 1, c \rangle$; การหมุนเป็นผลคูณ rotation matrix $R(\alpha) \cdot M$ ครั้งเดียวต่อการเลี้ยว	1, 2
ใช้ค่า ϵ ในการเดิน สิ้นเปลืองการคำนวณ	DDA (Digital Differential Analyzer)	สมการเส้นตรง $q(t) = p + t \cdot r$; ลำดับเลขคณิตสองลำดับ	3

ส่วนที่เหลือของภาพใช้เครื่องมือชุดเดียวกัน: ระยะถึงกำแพงมาจาก dot product และ ความสูงมาจากสามเหลี่ยมคล้าย (หัวข้อ 4) ส่วนการเดินเป็น linear combination ของ basis vector ของกล้อง (หัวข้อ 5)

1. Vector and Camera Matrix

กล้องเก็บอยู่ใน vector สองตัว และรังสีของทุกคอลัมน์เกิดจากการคูณ matrix หนึ่งครั้ง ผู้เล่นมีตำแหน่ง p (vector ในหน่วยช่อง) ทิศทาง d (vector หนึ่งหน่วย $|d| = 1$) และ รัศมกล้อง P ที่ตั้งฉากกับ d ยาว 0.66 ยิ่ง P ยาว ยิ่งมองได้กว้าง มุมมอง $FOV = 2 \arctan 0.66 \approx 66.8^\circ$ รัศม P ได้จากการหมุน d ไป 90° :

$$\vec{P} = 0.66 J \vec{d}, \quad J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

ความหมายของแต่ละตัวแปร:

- P: vector camera plane ตั้งฉากกับ d ชี้ไปทางขอบขวาของจอและยาว 0.66
- d: vector ทิศทางของผู้เล่นยาว 1 หน่วย
- 0.66: ค่าคงที่ที่กำหนดความยาวของ P ซึ่งกำหนดมุมมอง FOV = $2 \arctan 0.66 \approx 66.8^\circ$ ค่านี้มาก มองได้ยิ่งกว้าง (ใช้ 0.66 ตาม Wolfenstein 3D)
- J: rotation matrix 90° ส่ง vector $\langle x, y \rangle$ ไปที่ $\langle -y, x \rangle$

ตัวอย่าง $d = \langle 1, 0 \rangle$ ได้ $Jd = \langle 0, 1 \rangle$ และ $P = 0.66 \cdot \langle 0, 1 \rangle = \langle 0, 0.66 \rangle$

คอลัมน์ x ของจอกว้าง W พิกเซลได้พิกัด $c = 2x/W - 1$ ซึ่งอยู่ระหว่าง -1 (ขอบซ้าย) ถึง +1 (ขอบขวา) รังสีของคอลัมน์นั้นคือเดินหน้า 1 หน่วยตาม d แล้วเดินข้างอีก c เท่าของ P:

$$\vec{r} = \vec{d} + c \vec{P} = \underbrace{\begin{bmatrix} \vec{d} & \vec{P} \end{bmatrix}}_M \begin{bmatrix} 1 \\ c \end{bmatrix}$$

ความหมายของแต่ละตัวแปร:

- r: vector ทิศทางของรังสีสำหรับคอลัมน์นั้น (ทิศที่รังสีพุ่งไปบนแผนที่)
- d, P: vector ทิศทางของผู้เล่นและระนาบกล้อง (ตามที่อธิบายข้างบน)
- c: ตำแหน่งของคอลัมน์บนจอ จาก -1 (ซ้ายสุด) ถึง +1 (ขวาสุด) ค่า $c = 0$ คือกลางจอ
- x: ลำดับของคอลัมน์บนจอ นับจากซ้าย (0 ถึง $W - 1$)
- W: จำนวนคอลัมน์ของจอ (ความกว้างเป็นพิกเซล)
- M: matrix 2×2 ที่คอลัมน์แรกคือ d และคอลัมน์ที่สองคือ P
- $\langle 1, c \rangle$: พิกัดในระบบกล้อง คือไปข้างหน้า 1 หน่วย และไปข้าง c หน่วย

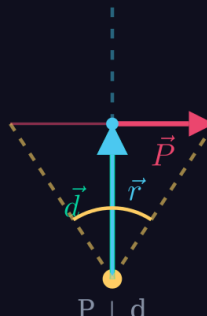
matrix M เก็บ d และ P ไว้เป็นสองคอลัมน์ และทำหน้าที่เป็น transformation matrix ที่แปลงตำแหน่งบนจอ c (ช่วง -1 ถึง +1) ให้เป็นทิศทางของรังสีบนแผนที่ โดยอิงจากมุมมองของผู้เล่น

ตัวอย่างจริง: ผู้เล่นหันไปทางตะวันออก $d = \langle 1, 0 \rangle$ จึงได้ $P = \langle 0, 0.66 \rangle$ คอลัมน์ $x = 1040$ ของจอกว้าง 1280 ได้ $c = 0.625$:

$$M \begin{bmatrix} 1 \\ 0.625 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0.66 \end{bmatrix} \begin{bmatrix} 1 \\ 0.625 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.4125 \end{bmatrix} = \vec{r}$$

การสร้างรังสีไม่ต้องใช้ sin หรือ cos เลย ใช้แค่การคูณและการบวก

The camera is a 2-column matrix



$$\vec{r} = \vec{d} + c\vec{P}$$

$$= \begin{bmatrix} \vec{d} & \vec{P} \end{bmatrix} \begin{bmatrix} 1 \\ c \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0.66 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ c \end{bmatrix}$$

N-facing: $d=(0, -1)$, $P=(0.66, 0)$

$c = +0.00$ $x = 640$

$r = (+0.00 , -1.00)$

$FOV = 66.8^\circ$ $|\vec{P}| = 0.66$

$$FOV = 2 \arctan \frac{|\vec{P}|}{|\vec{d}|}$$

x = 0 screen columns x = W-1

2. การหมุน: orthogonal matrix

การหมุนคือการคูณ M ด้วย rotation matrix

$$M' = R(\alpha) M, \quad R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix}$$

ความหมายของแต่ละตัวแปร:

- M: matrix กล้องก่อนหมุน (คอลัมน์คือ d และ P)
- M': matrix กล้องหลังหมุน (คอลัมน์คือ d และ P ใหม่)
- $R(\alpha)$: rotation matrix ที่หมุนไปมุม α
- α : มุมที่หมุนในเฟรมนี้ หน่วยเรเดียน โดย $\alpha = dt \cdot ROT_SPEED$ (dt คือเวลาจากเฟรมก่อน หน่วยวินาที)
- $\cos \alpha$, $\sin \alpha$: สองค่าที่ต้องคำนวณ ซึ่งคำนวณครั้งเดียวต่อการหมุนหนึ่งครั้ง

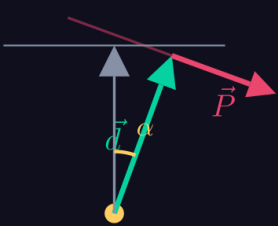
R เป็น orthogonal matrix: $R^T R = I$ และ $\det R = 1$ จึงหมุนโดยไม่ยืดและไม่สะท้อน
 หลังหมุนก็ครั้งก็ยังได้ $|d| = 1$, $|P| = 0.66$ และ $d \cdot P = 0$ ตัวอย่าง หันขึ้นเหนือแล้วหมุน
 $\alpha = 0.5$ เรเดียน:

$$R(0.5) \begin{bmatrix} 0 & 0.66 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} 0.4794 & 0.5792 \\ -0.8776 & 0.3164 \end{bmatrix}$$

โปรแกรมเรียก sin และ cos ของ α เพียงครั้งเดียวต่อการหมุนหนึ่งครั้ง แล้วนำ d กับ P
 ใหม่ไปใช้กับรังสีทั้ง 1280 เส้น

Turning = one matrix product on both columns

$$R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix}$$

$$\underbrace{\begin{bmatrix} d'_x & P'_x \\ d'_y & P'_y \end{bmatrix}}_{\text{new camera}} = R(\alpha) \underbrace{\begin{bmatrix} d_x & P_x \\ d_y & P_y \end{bmatrix}}_{\text{old camera}}$$


player_rotate() (C):

```

dir_x = old_dir_x*cos - dir_y*sin
dir_y = old_dir_x*sin + dir_y*cos
plane_x = old_plane_x*cos - plane_y*sin
plane_y = old_plane_x*sin + plane_y*cos
    
```

$\alpha = 0.35\text{rad}$

$|\vec{d}|, |\vec{P}|$ unchanged $\vec{d} \cdot \vec{P} = 0$ still

3. DDA (Digital Differential Analyzer)

รังสีเป็นเส้นตรง จุดบนรังสีที่ระยะ t คือ $q(t) = p + t \cdot r$ การหาว่ารังสีชนกำแพงที่ไหนคือ
 การเดินไปตามเส้นนี้ทีละช่อง (grid) จนถึงช่องที่เป็นกำแพง

$$\vec{q}(t) = \vec{p} + t \vec{r}$$

ความหมายของแต่ละตัวแปร:

- $q(t)$: จุดบนรังสี (vector ตำแหน่งบนแผนที่) ที่ค่า t
- t: ระยะที่เดินไปตามรังสี นับเป็นหน่วยของ r เนื่องจากส่วนที่ไปข้างหน้าของ r ยาว 1
 ค่า t จึงเท่ากับ perpendicular distance ด้วย

- p: ตำแหน่งผู้เล่น ซึ่งเป็นจุดเริ่มต้น $q(0) = p$
- r: vector ทิศทางของรังสี

การเดิน 1 ช่องมีการเดินสองแบบแยกกัน คือเดินในแกน x และเดินในแกน y เส้นกริดอยู่ที่จำนวนเต็ม จึงต้องรู้ว่าเดิน 1 ช่องในแต่ละแกนใช้ t เพิ่มเท่าไร ซึ่งได้จากสมการเส้นตรง โดยแยกเป็นแต่ละแกน จัดรูปให้เป็น t แล้ว diff เทียบกับพิกัด:

$$q_x = p_x + t r_x \implies t = \frac{q_x - p_x}{r_x} \implies \frac{dt}{dq_x} = \frac{1}{r_x}$$

นั่นคือเดิน 1 ช่องในแกน x ต้องเพิ่ม t เท่ากับ $1/|r_x|$ (ใช้ค่าสัมบูรณ์เพื่อให้เป็นบวกเสมอ แม้รังสีจะไปทางซ้ายหรือขึ้น) แกน y ก็ทำนองเดียวกัน:

$$\Delta t_x = \frac{1}{|r_x|}, \quad \Delta t_y = \frac{1}{|r_y|}$$

ความหมายของแต่ละตัวแปร:

- $\Delta t_x, \Delta t_y$: ค่า t ที่ต้องเพิ่มเพื่อให้รังสีเลื่อนไป 1 ช่องในแกน x หรือแกน y (คือระยะห่างของ t ระหว่างการตัดเส้นตั้งสองเส้นติดกัน หรือเส้นนอนสองเส้นติดกัน)
- r_x, r_y : องค์ประกอบแกน x และแกน y ของ r

ดังนั้นแต่ละแกนมีค่า t ที่รังสีชนเส้นกริดเรียงเป็นลำดับเลขคณิต: $s_x + k \cdot \Delta t_x$ และ $s_y + k \cdot \Delta t_y$ ($k = 0, 1, 2, \dots$) การเดินเริ่มต้นคือเดินไปถึงเส้นกริดเส้นแรกของแต่ละแกนก่อน ซึ่งได้ค่า s_x, s_y คือส่วนของช่องที่ยังอยู่ข้างหน้าคุณ Δt (m คือช่องที่ผู้เล่นยืนอยู่ คือ p ปัดลง):

$$s_x = \begin{cases} (p_x - m_x) \Delta t_x & r_x < 0 \\ (m_x + 1 - p_x) \Delta t_x & r_x \geq 0 \end{cases}$$

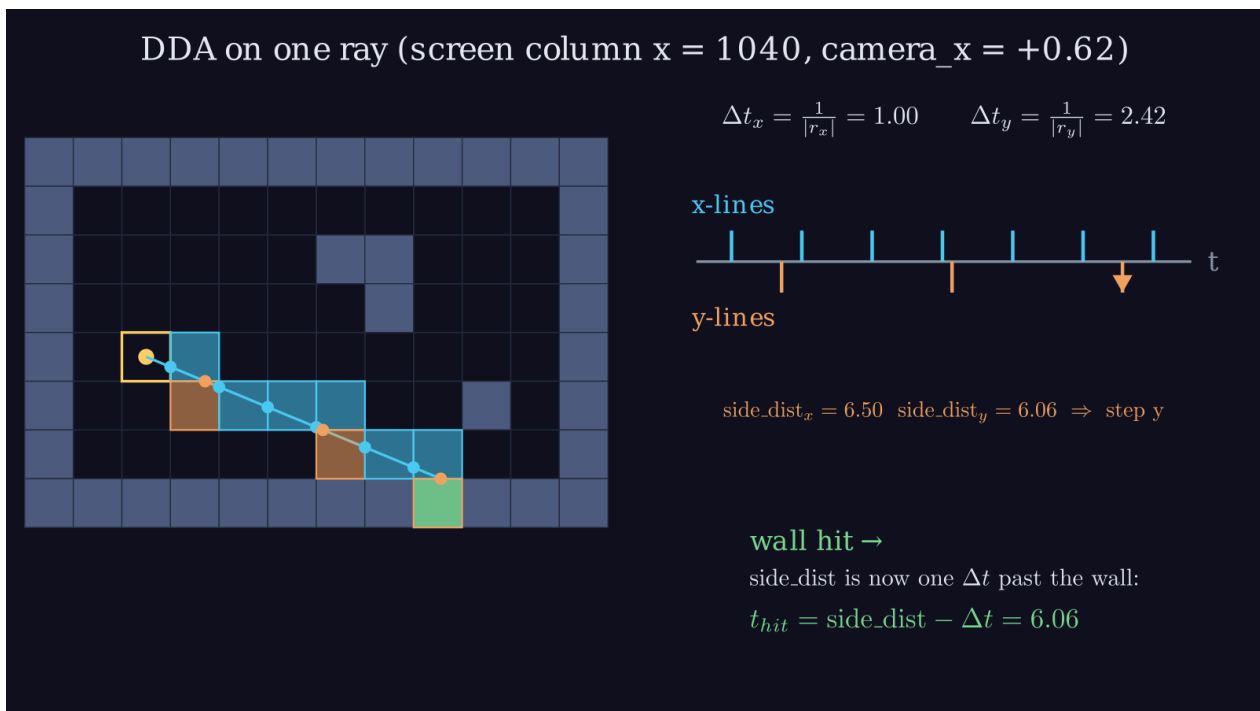
ความหมายของแต่ละตัวแปร:

- s_x : ค่า t ที่รังสีชนเส้นตั้งเส้นแรก (s_y เป็นค่าของเส้นนอนในทำนองเดียวกัน)
- p_x : พิกัด x ของผู้เล่น
- m_x : เลขช่องแนว x ที่ผู้เล่นยืนอยู่ คือ p_x ปัดลง
- กรณี $r_x < 0$: รังสีไปทางซ้าย เส้นแรกคือขอบซ้ายของช่อง ห่างออกไป $p_x - m_x$
- กรณี $r_x \geq 0$: รังสีไปทางขวา เส้นแรกคือขอบขวาของช่อง ห่างออกไป $m_x + 1 - p_x$

- การคูณด้วย Δt_x : แปลงระยะบนแผนที่ (หน่วยช่อง) ให้เป็นค่า t

จากนั้น DDA เดินทีละรอบ ทุกรอบเดินเพียงแกนเดียว: เปรียบเทียบค่า t ปัจจุบันของแกน x กับแกน y ถ้าฝั่งหนึ่งมากกว่าให้เดินอีกฝั่ง (เดินฝั่งที่ค่าน้อยกว่า) แล้วบวก Δt ของฝั่งนั้นเพิ่ม เข้าช่องถัดไปและเช็คว่าเป็นกำแพงหรือไม่ เดินไปเรื่อยๆ จนกว่าจะถึงกำแพง และเพราะทุกช่องที่รังสีผ่านถูกเช็คครบ จึงไม่มีกำแพงบางใดหลุดรอดไปได้

ตัวอย่างจากรังสีเดียวกับหัวข้อ 1 ($p = \langle 2.5, 4.5 \rangle$, $r = \langle 1, 0.4125 \rangle$) ได้ $\Delta t_x = 1.00$ และ $\Delta t_y = 2.42$ DDA ใช้ 9 รอบก็ถึงกำแพงที่ช่อง (8, 7) แทนที่จะเดินทีละ 0.1 ประมาณ 60 ก้าว ได้ $t = 6.06$



4. Projection to 3D

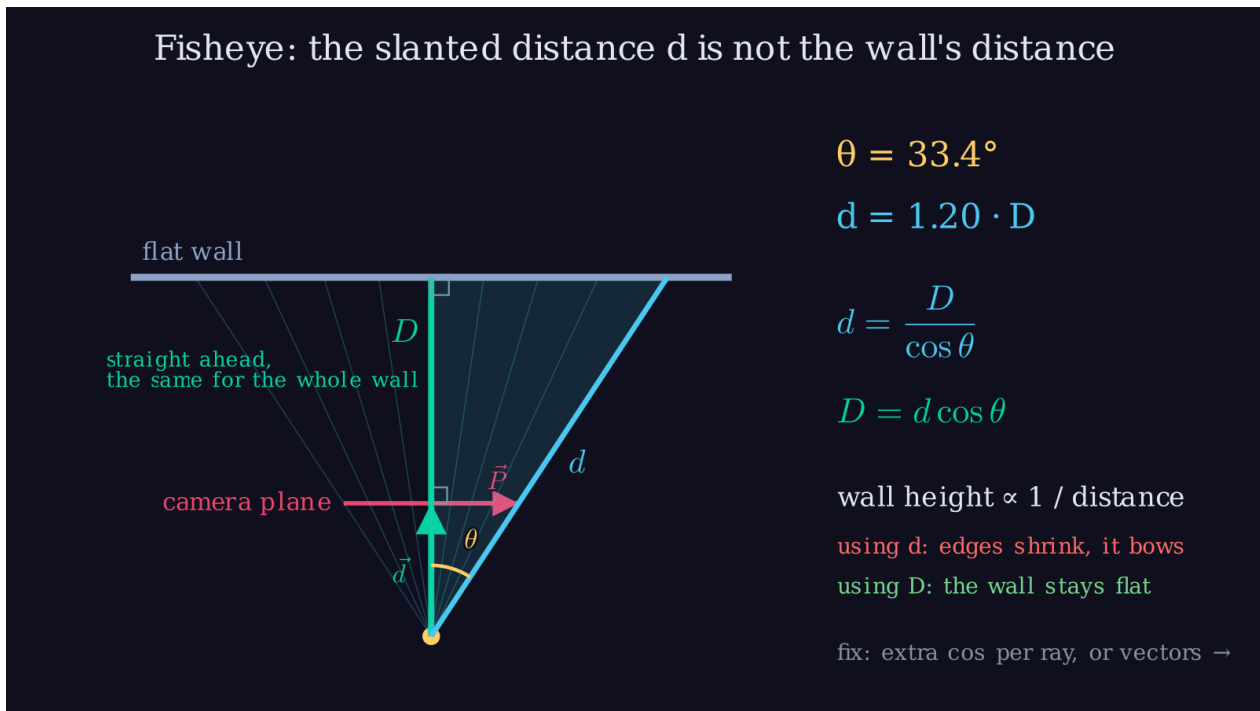
ค่า t ที่ได้จากการเดินคือ perpendicular distance ถึงระนาบกล้องอยู่แล้ว ไม่ต้องคูณ $\cos$ เพิ่ม

ความสูงของกำแพงบนจอควรขึ้นกับระยะไปข้างหน้า D (ตามทิศ d) ไม่ใช่ความยาวเส้นเฉียงตามรังสี ถ้าใช้ระยะเฉียง กำแพงเรียบจะดูโค้งคล้ายภาพตาปลา รังสีที่เบนจากทิศ d ไปมุม θ ยาวกว่า D อยู่ $1/\cos \theta$ เท่า (ตัว d ในสูตรนี้และในภาพคือความยาว ไม่ใช่ vector ทิศทาง d):

$$d = \frac{D}{\cos \theta} \iff D = d \cos \theta$$

ความหมายของแต่ละตัวแปร:

- D: perpendicular distance จากผู้เล่นถึงกำแพง วัดตามทิศ d (ไปข้างหน้า)
- d: ระยะเฉียงตามรังสีถึงกำแพง (เป็นความยาว ไม่ใช่ vector)
- θ : มุมระหว่างรังสีกับทิศ d



แต่เราไม่ต้องคำนวณ cos เพราะรังสี $r = d + cP$ มีส่วนไปข้างหน้า (ตาม d) ยาว 1 เสมอ ส่วน cP ตั้งฉากกับ d จึงไม่ช่วยให้ไปข้างหน้าเพิ่ม dot product บอกสิ่งนี้ได้ ตรงๆ คือส่วนของ vector ที่ชี้ไปตาม d (เพราะ $|d| = 1$):

$$\vec{d} \cdot (t \vec{r}) = t (\vec{d} \cdot \vec{d} + c \vec{d} \cdot \vec{P}) = t (1 + c \cdot 0) = t$$

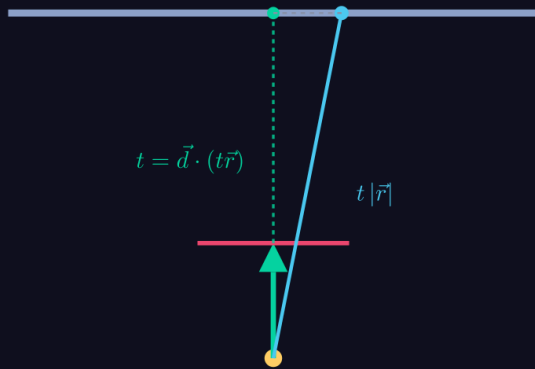
ความหมายของแต่ละตัวแปร:

- $\therefore$ dot product ของ vector สองตัว
- t r: vector จากผู้เล่นไปถึงจุดที่รังสีชนกำแพง
- $d \cdot d = 1$: เพราะ d ยาว 1 หน่วย
- $d \cdot P = 0$: เพราะ d ตั้งฉากกับ P
- ผลลัพธ์ t: ส่วนของ t r ที่ชี้ไปข้างหน้าตาม d ซึ่งคือ perpendicular distance

เพราะ $d \cdot d = 1$ และ $d \cdot P = 0$ (ตั้งฉากกัน) จึงได้ $d \cdot r = 1$ สำหรับทุกคอลัมน์ perpendicular distance จึงเท่ากับ t_hit พอดี เรียกค่านี้ว่าความลึก $z = t_hit$ ตรงกับที่เช็คด้วยมุมได้: $|r| = 1/\cos \theta$ จึง $t|r| \cdot \cos \theta = t$

Perpendicular distance comes for free

$$\vec{d} \cdot \vec{r} = \vec{d} \cdot \vec{d} + c(\vec{d} \cdot \vec{P}) = 1 + 0 = 1$$



$$\text{Euclid } t|\vec{r}| = \frac{t}{\cos \theta} \quad \text{perp} = \frac{\vec{d} \cdot (t\vec{r})}{|\vec{d}|} = t$$

$$\text{wall_dist} = \text{side_dist} - \Delta t = t$$

no cos, no sqrt, no extra division

camera_x c = +0.30

|r| = 1.019

Euclid t|r| = 3.058

perp t = 3.000

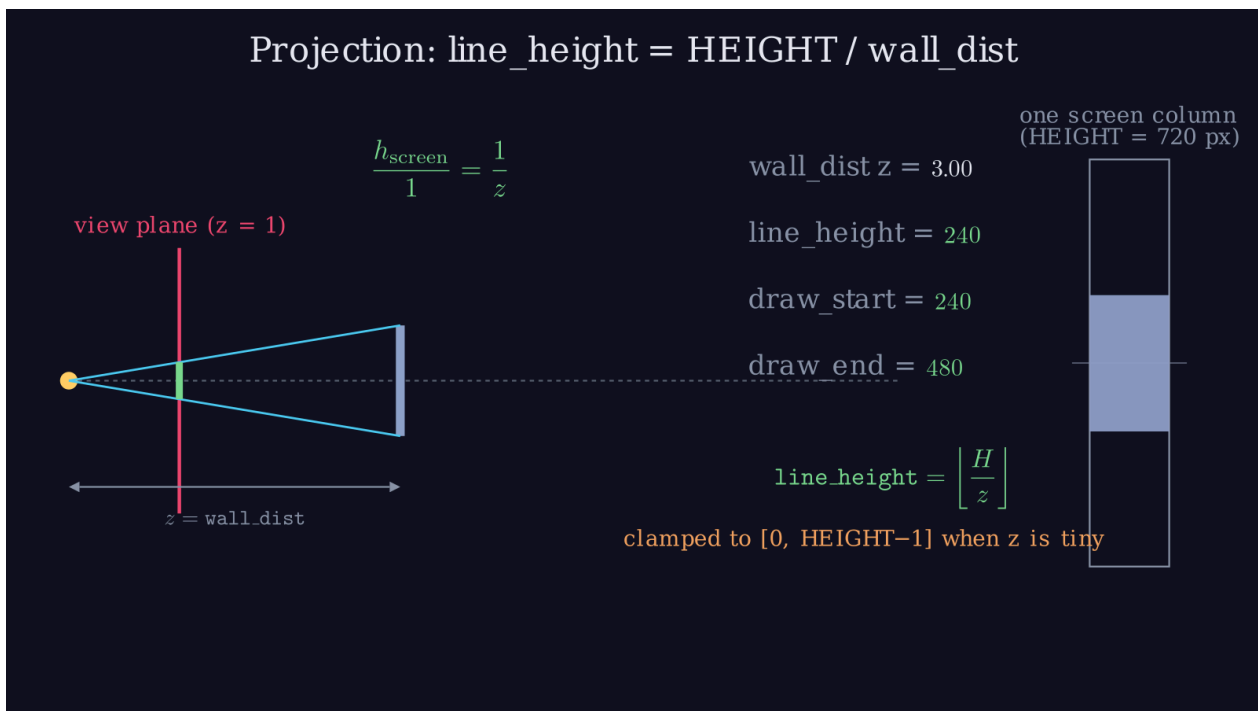
จากความลึก z ไปเป็นความสูงบนจอใช้สามเหลี่ยมคล้าย กำแพงสูง 1 ที่ความลึก z ไปปรากฏสูง $1/z$ บนระนาบที่ความลึก 1 เมื่อกำหนดให้ความสูง 1 บนระนาบเท่ากับ H พิกเซล เราได้ความสูงกำแพงในหน่วยพิกเซล (และจัดให้อยู่กึ่งกลางจอ):

$$lh = \left\lfloor \frac{H}{z} \right\rfloor, \quad \text{draw_start} = \frac{H}{2} - \frac{lh}{2}, \quad \text{draw_end} = \frac{H}{2} + \frac{lh}{2}$$

ความหมายของแต่ละตัวแปร:

- z : ความลึก (perpendicular distance ถึงกำแพง) โดย $z = t_{\text{hit}}$
- H : ความสูงของจอเป็นพิกเซล (ตัวอย่างใช้ 720)
- lh : ความสูงของเส้นกำแพงในคอลัมน์นั้น (พิกเซล) และเครื่องหมาย $\lfloor \rfloor$ คือการปัดลงเป็นจำนวนเต็ม
- $\text{draw_start}, \text{draw_end}$: แถวบนสุดและล่างสุดที่ใช้วาดกำแพง นับจากขอบบนของจอ โดยจัดให้อยู่กึ่งกลางที่ $H/2$

ตัวอย่าง $H = 720$: $z = 3$ ได้ 240 แถว รังสีในหัวข้อ 3 ($z = 6.06$) ได้ 118 แถว



ทิศทางตรงข้าม: inverse matrix

การสังรังสีคือการกลับชั้นของ projection จากจุด q ในโลกไปคอลัมน์บนจอ ใช้ inverse matrix M^{-1} แปลงเป็นพิกัดกล้อง (a คือไปข้างหน้า b คือไปข้าง) แล้วหารด้วยความลึก:

$$\begin{bmatrix} a \\ b \end{bmatrix} = M^{-1}(\vec{q} - \vec{p}), \quad c = \frac{b}{a}, \quad x = \frac{W}{2} (1 + c)$$

ความหมายของแต่ละตัวแปร:

- q : จุดในโลก (เช่น มุมของกำแพง) ที่ต้องการรู้ว่าจะปรากฏที่คอลัมน์ไหนของจอ
- p : ตำแหน่งผู้เล่น และ $q - p$ คือ vector จากผู้เล่นไปยังจุด q
- M^{-1} : inverse matrix ของ M ซึ่งแปลงทิศบนแผนที่กลับเป็นพิกัดกล้อง
- a, b : พิกัดกล้องของ $q - p$ โดย a คือไปข้างหน้า และ b คือไปข้าง
- c : ตำแหน่งของคอลัมน์ในช่วง -1 ถึง $+1$
- x : คอลัมน์ของจอ (พิกเซล)
- W : ความกว้างของจอเป็นพิกเซล

การหาร $c = b/a$ ทำให้ของที่อยู่ใกล้ดูเล็กลง การสังรังสีคือการทำทางตรงข้ามนี้: เริ่มจาก c แล้วสร้าง $r = M^{-1}(c)$ จากนั้นเดินทางจุดที่รังสีชน จุด $q = p + t \cdot r$ มี $a = t$ และ $b = t \cdot c$ ซึ่งเป็นเหตุผลเดียวกับที่ความลึกเท่ากับ t

5. การเคลื่อนที่ของผู้เล่น

การเดินคือ linear combination ของ basis vector ของกล้อง d กับ Jd ในทุกเฟรม ให้ dt เป็นเวลาจากเฟรมก่อน ระยะที่เดินในเฟรมนี้คือ $s = dt \cdot \text{MOVE_SPEED}$ (3.0 ช่องต่อวินาที จึงไม่ขึ้นกับความเร็วเฟรม) ปุ่ม W/S เดินไปหน้าหรือถอยหลังตาม d ปุ่ม D/A เดินข้างตาม Jd :

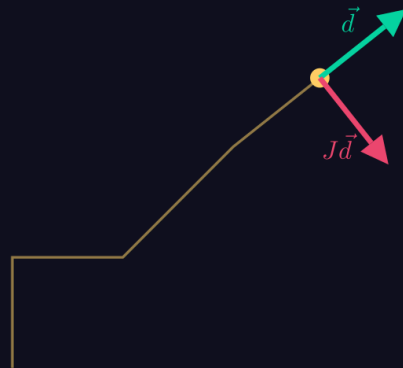
$$\text{W/S: } \vec{p} \leftarrow \vec{p} \pm s \vec{d}, \quad \text{D/A: } \vec{p} \leftarrow \vec{p} \pm s J\vec{d}$$

ความหมายของแต่ละตัวแปร:

- p : ตำแหน่งผู้เล่น
- ลูกศร $\leftarrow$ หมายถึงกำหนดค่าใหม่ให้ตัวแปรทางซ้าย
- s : ระยะที่เดินในเฟรมนี้ = $dt \cdot \text{MOVE_SPEED}$
- $\pm$: เครื่องหมาย + เมื่อกด W หรือ D (ไปหน้า/ไปข้างตาม Jd) และเครื่องหมาย - เมื่อกด S หรือ A (ถอยหลัง/ไปทางตรงข้าม)
- d : vector ทิศทางของผู้เล่น (ไปหน้า)
- Jd : d ที่ถูกหมุน 90° (ไปข้าง)

d กับ Jd ตั้งฉากกันและยาวเท่ากัน จึงเป็น orthonormal basis ของระนาบ และ $Jd = P/0.66$ จึงขนานกับระนาบกล้อง การกดสองปุ่มพร้อมกัน เช่น W+D ได้ $s(d + Jd)$ ซึ่งยาว $\sqrt{2} \cdot s$ เร็วกว่าเดินตรงประมาณ 41% เพราะโปรแกรมไม่ปรับผลรวมให้เป็น vector หนึ่งหน่วย

Moving = adding multiples of the camera basis



$$W: \vec{p} \leftarrow \vec{p} + s \vec{d}$$

$$D: \vec{p} \leftarrow \vec{p} + s J \vec{d}, \quad J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$\vec{P} = 0.66 J \vec{d} \Rightarrow \text{strafe} \parallel \text{camera plane}$$

$$W+D: |s(\vec{d} + J \vec{d})| = \sqrt{2} s \quad (\text{not normalised})$$

$$s = \Delta t \cdot \text{MOVE_SPEED} \quad (3.0 \text{ cells/s})$$

การชนกำแพง: ผู้เล่นเป็นสี่เหลี่ยมเล็กๆ โปรแกรมตรวจสอบ $(x \pm 0.2, y \pm 0.2)$ และทดลองเดินทีละแกน จึงไถตามกำแพงได้ไม่ติด

หมายเหตุ: โป้เจกต์นี้พัฒนาด้วยภาษา C และ OpenGL เพื่อให้สภาพแวดล้อมใกล้เคียงกับยุคของ Wolfenstein 3D (ปี 1992) ไม่พึ่ง framework สำเร็จรูป คณิตศาสตร์ทั้งหมดข้างต้นจึงต้องเขียนเองทุกขั้นตอน